UNIVERSITAS MULTI DATA PALEMBANG
Fakultas Ilmu Komputer dan Rekayasa
Program Studi Sistem Informasi

TEMPLAT LAPORAN

TUGAS 2

Postman Collection dan Automated Test Script

Pengujian *RESTful API* dengan *Postman Automation*

Pengembangan Aplikasi Web II | SUB-CPMK-7

Nama dan NIM	Sunita Aqila 2428240072
Kelas	SI5B
Judul project / topik API	RESTful API Ruang Kelas — No. 16 Kampus
Tautan repositori	https://github.com/oreo220/si5B_kuis1_SunitaAqila
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	Kamis, 22 Oktober 2026

Petunjuk Pengisian

Dokumen ini adalah laporan **Tugas 2**: menguji RESTful API project akhir Anda dengan Postman. Seluruh isinya menggunakan **API milik Anda sendiri**, bukan API Mahasiswa pada contoh materi. Pelajari langkahnya pada materi Pertemuan 8 sebelum mengisi.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya [sumber-daya], dengan data API Anda.
2. Bukti berupa gambar (*screenshot*) diisi pada kotak bergaris putus-putus. Cuplikan kode test script **ditempel sebagai teks**, bukan gambar.
3. Setiap bukti memiliki keterangan dan kotak **Penjelasan**. Tulis penjelasan dengan kalimat Anda sendiri dan sebut nama *request* atau fungsi dari API Anda.
4. Simpan hasil akhir sebagai PDF, lalu kumpulkan melalui LMS sesuai ketentuan dosen (format nama berkas dan batas waktu mengikuti petunjuk di LMS).

B. Bukti yang diminta

No	Bukti	Bagian	Jenis
1	Daftar request dalam Collection	Bab 2	Screenshot (wajib)
2	Cuplikan test script	Bab 3	Kode teks (wajib)
3	Ringkasan Collection Runner	Bab 4	Screenshot (wajib)
4	Hasil Newman di terminal	Bab 5	Screenshot (pengayaan)

C. Syarat tangkapan layar

Bukti	Yang harus terlihat
Collection	Nama Collection, kedua folder (bila ada), dan seluruh nama request. Buka semua folder sebelum mengambil gambar.
Ringkasan Runner	Nama Collection, Environment yang dipakai, jumlah request, jumlah test **lulus** dan **gagal**. Seluruh test harus lulus (0 gagal).
Newman	Perintah yang diketik, daftar hasil, dan tabel executed dan failed di bagian akhir.
Larangan	Gambar dipotong sehingga angka hasil tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Sebelum mengambil screenshot Runner
Nyalakan ulang server agar data kembali ke kondisi awal, pilih Environment yang benar, lalu jalankan seluruh Collection. Bila ada test gagal, perbaiki dulu dan jalankan ulang sampai 0 gagal.

Bab 1 Identitas API dan Daftar Endpoint

Bab ini menjelaskan API yang diuji, agar pembaca memahami cakupan pengujian pada bab berikutnya.

1.1 Deskripsi singkat API

Penjelasan:
RESTful API Ruang Kelas merupakan API yang digunakan untuk mengelola data ruang kelas, meliputi kode ruang, gedung, lantai, kapasitas, dan ketersediaan proyektor. API ini dikembangkan menggunakan Express.js dengan arsitektur backend terstruktur yang memisahkan Route, Controller, Model, dan Middleware. Pengujian dilakukan menggunakan Postman untuk memastikan setiap endpoint menghasilkan response dan HTTP status code sesuai dengan ketentuan.

1.2 Daftar endpoint

Tuliskan **seluruh** *endpoint* API Anda. Kolom "API key" diisi Ya bila rute dilindungi cekApiKey atau mekanisme serupa.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/classrooms	Tidak	200	Mengambil seluruh data ruang kelas
2	GET	/classrooms/:id	Tidak	200	Mengambil data berdasarkan ID
3	POST	/classrooms	Ya	201	Menambahkan data ruang kelas
4	PUT	/classrooms/:id	Ya	200	Memperbarui data ruang kelas
5	DELETE	/classrooms/:id	Ya	204	Menghapus data ruang kelas

API key diwajibkan pada operasi POST, PUT, dan DELETE melalui middleware cekApiKey. Request GET tidak memerlukan API key.

1.3 Variabel Environment

Environment Postman yang digunakan bernama development.

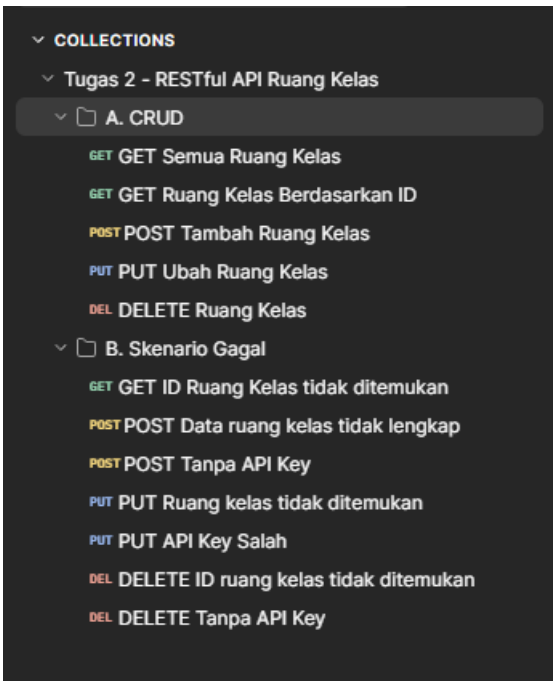
No	Nama variabel	Nilai (contoh)	Fungsi
1	base_url	http://localhost:3000	Alamat server
2	api_key	Kunci_Sunita	Nilai header kunci API
3	classroomId	Kosong Pada Awal Pengujian	Menyimpan ID dari respons POST

Bab 2 Postman Collection

Bab ini membuktikan bahwa seluruh *endpoint* sudah dikelompokkan dalam satu Collection.

2.1 Daftar request dalam Collection

Nama Collection yang digunakan adalah Tugas 2 - RESTful API Ruang Kelas. Request dikelompokkan berdasarkan fungsi agar pengujian lebih teratur.



Gambar 1. Collection beserta seluruh request

Penjelasan:
Saya mengelompokkan request ke dalam folder CRUD dan Skenario Gagal agar pengujian lebih mudah dipahami. Folder CRUD digunakan untuk menguji fungsi utama API, sedangkan folder Skenario Gagal digunakan untuk memeriksa penanganan kesalahan, seperti API key yang tidak valid, body yang tidak lengkap, JSON yang rusak, dan ID yang tidak ditemukan.

2.2 Tabel request

Salin daftar request pada gambar di atas ke tabel berikut. Setiap *endpoint* pada Bab 1 harus punya minimal satu request.

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
1	GET Semua Ruang Kelas	GET	{{base_url}}/classrooms	A. CRUD
2	GET Ruang Kelas Berdasarkan ID	GET	{{base_url}}/classrooms/1	A. CRUD
3	POST Tambah Ruang Kelas	POST	{{base_url}}/classrooms	A. CRUD
4	PUT Ubah Ruang Kelas	PUT	{{base_url}}/classrooms/{{classroomId}}	A. CRUD

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
5	DELETE Ruang Kelas	DELETE	{{base_url}}/classrooms/{{classroomId}}	A. CRUD
6	GET ID Ruang Kelas tidak ditemukan	GET	{{base_url}}/classrooms/99	B. Skenario Gagal
7	POST Data ruang kelas tidak lengkap	POST	{{base_url}}/classrooms	B. Skenario Gagal
8	POST Tanpa API Key	POST	{{base_url}}/classrooms	B. Skenario Gagal
9	PUT Ruang kelas tidak ditemukan	PUT	{{base_url}}/classrooms/99	B. Skenario Gagal
10	PUT API Key Salah	PUT	{{base_url}}/classrooms/{{classroomId}}	B. Skenario Gagal
11	DELETE ID ruang kelas tidak ditemukan	DELETE	{{base_url}}/classrooms/99	B. Skenario Gagal
12	DELETE Tanpa API Key	DELETE	{{base_url}}/classrooms/{{classroomId}}	B. Skenario Gagal

Bab 3 Automated Test Script

Bab ini menunjukkan *assertion* yang Anda tulis. Setiap request wajib memiliki minimal satu *assertion* status code dan satu *assertion* struktur atau isi data JSON.

3.1 Rekap assertion per request

Setiap request dilengkapi dengan assertion untuk memeriksa HTTP status code dan struktur atau isi response. Untuk request DELETE yang berhasil, pemeriksaan dilakukan terhadap status 204 dan response body yang kosong.

No	Nama request	Assertion status code	Assertion struktur atau isi JSON
1	GET Semua Ruang Kelas	Status 200	Response berupa array dan memuat minimal tiga data awal
2	GET Ruang Kelas Berdasarkan ID	Status 200	Memeriksa field id, kodeRuang, gedung, dan kapasitas
3	POST Tambah Ruang Kelas	Status 201	Memeriksa ID dan kode ruang pada data baru serta menyimpan ID ke Environment
4	PUT Ubah Ruang Kelas	Status 200	Memeriksa data setelah perubahan, termasuk kapasitas
5	DELETE Ruang Kelas	Status 204	Memastikan response body kosong
6	GET ID Ruang Kelas tidak ditemukan	Status 404	Memeriksa pesan bahwa ruang kelas tidak ditemukan
7	POST Data ruang kelas tidak lengkap	Status 400	Memeriksa pesan validasi field wajib
8	POST Tanpa API Key	Status 401	Memeriksa pesan API key tidak valid
9	PUT Ruang kelas tidak ditemukan	Status 404	Memeriksa pesan bahwa ruang kelas tidak ditemukan
10	PUT API Key Salah	Status 401	Memeriksa pesan API key tidak valid
11	DELETE ID ruang kelas tidak ditemukan	Status 404	Memeriksa pesan bahwa ruang kelas tidak ditemukan
12	DELETE Tanpa API Key	Status 401	Memeriksa pesan API key tidak valid

3.2 Cuplikan 1: request GET

Nama Request : GET Semua Ruang Kelas

```
// Test script request GET: GET Semua Ruang Kelas
pm.test("Status code 200", function () {
    pm.response.to.have.status(200);
});

pm.test("Response berupa array", function () {
    pm.expect(pm.response.json()).to.be.an("array");
});

pm.test("Minimal ada 3 data awal", function () {
    pm.expect(pm.response.json().length).to.be.at.least(3);
});
```

Penjelasan:
Assertion pertama memeriksa apakah request GET Semua Ruang Kelas menghasilkan status code 200. Assertion kedua memastikan response berbentuk array karena endpoint mengembalikan kumpulan data ruang kelas. Assertion ketiga memastikan jumlah data minimal tiga record sesuai dengan data awal pada model API.

3.3 Cuplikan 2: request POST atau request yang menyimpan variabel

Nama Request : POST Tambah Ruang Kelas

```
Test script request POST: POST Tambah Ruang Kelas

pm.test("Status code 201", function () {
    pm.response.to.have.status(201);
});

pm.test("Data baru berhasil dibuat", function () {
    const data = pm.response.json();

    pm.expect(data).to.have.property("id");
    pm.expect(data.kodeRuang).to.eql("S-2428240072");
});

const data = pm.response.json();

pm.environment.set("classroomId", data.id);
```

Penjelasan:

Script memeriksa apakah request POST Tambah Ruang Kelas berhasil menghasilkan status code 201 dan data baru memiliki ID serta kode ruang yang sesuai. Setelah itu, ID dari response disimpan ke variabel Environment classroomId. Variabel tersebut digunakan oleh request PUT Ubah Ruang Kelas dan DELETE Ruang Kelas agar kedua request dapat mengakses data yang sama tanpa harus memasukkan ID secara manual.

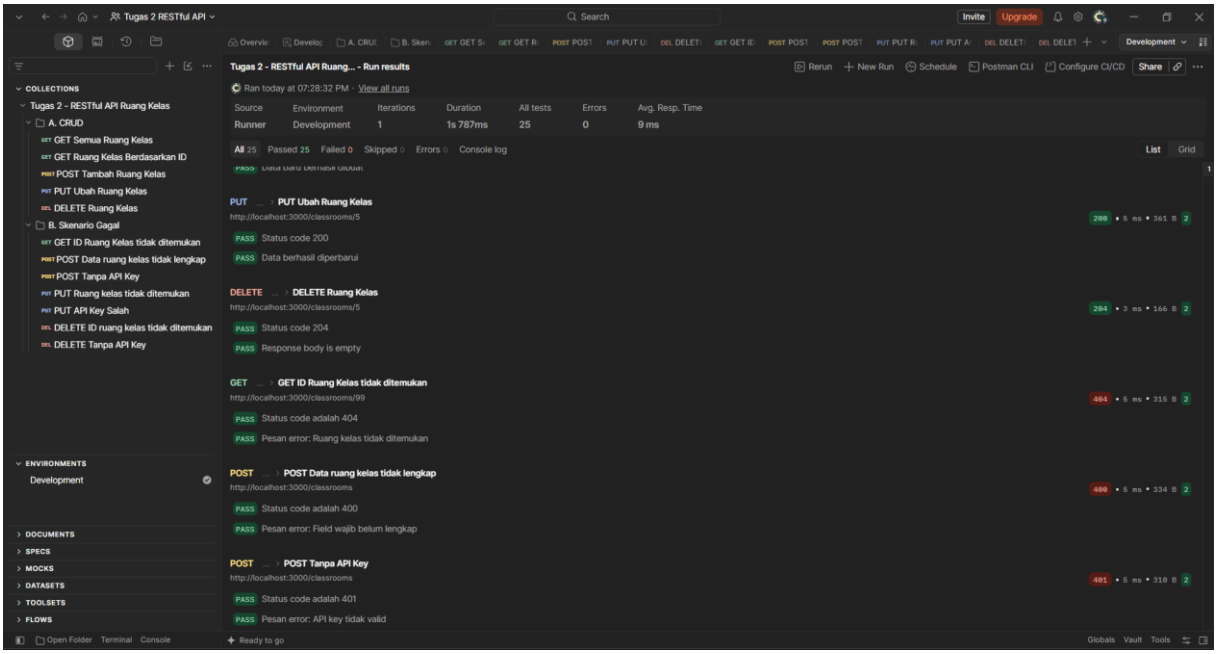
Urutan request berpengaruh karena POST harus berhasil terlebih dahulu agar classroomId memperoleh nilai yang benar. Jika urutan ditukar sehingga PUT atau DELETE dijalankan sebelum POST berhasil, variabel tersebut dapat kosong atau berisi ID sebelumnya sehingga request tidak berjalan sesuai skenario yang diharapkan.

Bab 4 Hasil Collection Runner

Bab ini adalah bukti utama bahwa seluruh test lolos saat Collection dijalankan sekaligus.

4.1 Ringkasan Collection Runner

Harus terlihat pada screenshot: nama Collection, Environment, jumlah request, jumlah test lulus dan gagal. Seluruh test harus lulus.



Gambar 2. Ringkasan hasil Collection Runner

Penjelasan:
Sebelum menjalankan Collection Runner, server Express.js dinyalakan kembali agar data kembali ke kondisi awal. Environment development dipilih, kemudian Collection dijalankan untuk memeriksa seluruh request dan assertion yang telah dibuat. Hasil pengujian dicatat berdasarkan jumlah test yang lulus dan gagal pada tampilan Runner.

4.2 Rekap angka hasil

Nama Collection	Tugas 2 – RESTful API Ruang Kelas
Environment yang dipakai	Development
Jumlah request dijalankan	12
Jumlah test lulus	25
Jumlah test gagal	0
Tanggal dan jam pengujian	19 Oktober 2026 , Jam 19:00 WIB

4.3 Temuan selama pengujian

Catat kegagalan yang pernah muncul sebelum akhirnya lulus. Bagian ini menunjukkan bahwa pengujian benar-benar membantu menemukan masalah. Bila tidak ada, tulis "tidak ada kegagalan".

No	Request	Kegagalan (pesan)	Penyebab	Perbaikan
1	POST Tambah Ruang Kelas	Status code tidak sesuai harapan	API key belum dicantumkan	Memeriksa header x-api-key dan menggunakan API key yang benar
2	PUT Ubah Ruang Kelas	Data tidak ditemukan	Variabel classroomId kosong atau tidak mengarah ke ID yang tersedia	Menjalankan POST berhasil terlebih dahulu dan memastikan ID tersimpan di Environment

Bab 5 Newman di Terminal (Pengayaan)

> **Bagian ini opsional**
> Bab ini adalah pengayaan dan tidak mengubah kriteria penilaian. Isi bila Anda menjalankan Collection dengan Newman. Bila tidak, tulis "tidak dikerjakan" di bawah judul ini.

5.1 Tidak Dikerjakan

Harus terlihat pada screenshot: perintah `newman run ...`, daftar hasil, dan tabel executed dan failed.

[Tempel screenshot di sini]

Gambar 3. Hasil menjalankan Collection dengan Newman

Gambar 3. Hasil menjalankan Collection dengan Newman

Penjelasan:

Tulis perintah yang Anda jalankan dan jelaskan arti exit code (0 atau 1) pada hasil Anda.

Bab 6 Repositori dan Berkas

Tautan repositori	https://github.com/oreo220/si5B_kuis1_SunitaAqila
Lokasi berkas Collection (.json)	C:\Postman sany\tugas2-collection.json
Lokasi berkas Environment (.json)	C:\Postman sany\development.json
Berkas Environment tidak memuat kunci rahasia sungguhan	[✓] Ya [ ] Tidak
Repositori dapat dibuka dosen	[✓] Ya [ ] Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan pengalaman Anda mengerjakan tugas ini.

Pertanyaan 1

Dari seluruh *assertion* yang Anda tulis, mana yang paling berguna untuk menemukan kesalahan pada API Anda? Berikan alasannya.

Penjelasan:
Menurut saya, assertion status code paling berguna karena saya bisa mengetahui apakah API sudah memberikan respons yang sesuai dengan request. Contohnya, saat menguji API key, saya bisa memastikan bahwa request tanpa API key atau dengan API key yang salah ditolak dengan status 401.

Pertanyaan 2

Apa perbedaan yang Anda rasakan antara menguji API secara manual dan memakai Collection Runner? Bagian mana yang paling sulit saat menulis test script, dan bagaimana Anda mengatasinya?

Penjelasan:
Menurut saya, pengujian manual lebih mudah dipahami karena saya bisa mengecek hasil setiap request secara langsung, sedangkan Collection Runner lebih praktis karena bisa menjalankan banyak request sekaligus. Bagian yang cukup sulit bagi saya adalah mengatur variabel classroomId, jadi saya perlu memastikan ID dari hasil POST tersimpan dengan benar agar bisa digunakan pada request PUT dan DELETE.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda [...] sudah diganti dengan data API saya	✓
2	Setiap endpoint API saya ada di Collection	✓
3	Alamat memakai {{base_url}}, bukan ditulis langsung	✓
4	Setiap request punya assertion status code dan struktur JSON	✓
5	Nama setiap test berupa kalimat yang menyatakan harapan	✓
6	POST menyimpan ID dengan pemeriksaan status, dan request berikutnya memakainya	✓
7	Server dinyalakan ulang sebelum Runner dijalankan	✓
8	Screenshot Runner menampilkan 0 test gagal dan angkanya terbaca jelas	✓
9	Berkas Environment tidak memuat kunci rahasia sungguhan	✓
10	Berkas disimpan sebagai PDF dan dikumpulkan sesuai ketentuan LMS	✓

Lampiran B Kriteria Penilaian

Kriteria berikut sesuai dengan slide materi Pertemuan 8. Bobot nilai mengikuti ketentuan dosen di LMS.

Kriteria	Deskripsi	Dinilai dari
Kelengkapan Postman Collection	Seluruh endpoint project akhir tercakup dalam satu Collection	Bab 1 dan Bab 2
Ketepatan automated test script	Assertion status code dan struktur response JSON tepat sasaran	Bab 3
Keberhasilan Collection Runner	Seluruh test lolos saat Collection dijalankan	Bab 4
Kejelasan dokumentasi hasil	Tangkapan layar ringkasan pengujian disertakan dengan jelas	Bab 4 dan seluruh dokumen